

“Hospital Management & Appointment System (MediDesk)”

A Project Report Submitted in partial fulfillment of the requirements of the course
**DATABASE SYSTEMS ENGINEERING & DISTRIBUTED BACKEND
DEVELOPMENT (25CS1302E)**

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

P. RAHUL

2520090016

A. LAKSHMI SAI TEJA

2520080005

Section 10 | Team 15 | Y25 · 2026–2027 · Trimester 4

Under the Esteemed Guidance of

[Guide Name]

[Designation]

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Declaration

The Project Report entitled “Hospital Management & Appointment System (MediDesk)” is a record of bonafide work of P. Rahul – 2520090016 and A. Lakshmi Sai Teja – 2520080005, submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 15, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

P. Rahul – 2520090016

A. Lakshmi Sai Teja – 2520080005

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project report entitled “Hospital Management & Appointment System (MediDesk)” is a bonafide work done and submitted by P. Rahul – 2520090016 and A. Lakshmi Sai Teja – 2520080005 in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 15, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us in one way or another in completing this project.

Our greatest appreciation goes to our Course Coordinator, [Course Coordinator Name], and our guide, [Guide Name], Department of Computer Science and Information Technology, for the tremendous support, encouragement and guidance they gave us throughout this project.

We express our gratitude to [HOD Name], Head of the Department of Computer Science and Information Technology, for providing us with the facilities and means by which we were able to complete this project-based lab.

We thank all the teaching and non-teaching staff members who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

P. Rahul – 2520090016

A. Lakshmi Sai Teja – 2520080005

ABSTRACT

The Hospital Management & Appointment System, branded **MediDesk**, is a web-based application that simplifies and digitises hospital operations through one centralised platform. Hospitals that still depend on paper registers and disconnected tools face long waiting times, inconsistent appointment tracking, data-entry errors and patient records that are hard to reach. Billing, pharmacy and laboratory work also tend to run in silos, so information is repeatedly re-entered and often out of date.

MediDesk addresses these problems with three role-based portals for **patients, doctors and administrators**. Patients register, book appointments against doctor availability, follow a live OPD token, and view their medical records, bills and diagnostic reports. Doctors work from a clinical console that shows today's appointments, the OPD queue, a consultation room and a patient directory. Administrators use a command-centre dashboard to monitor patients, doctors, appointments, revenue, the central OPD queue, pharmacy stock levels and laboratory work, and to review security audit logs.

The application uses a **React.js (Vite)** frontend, a **Node.js and Express.js** REST backend organised into routes, controllers and models, and a **MySQL** database accessed through the **Sequelize** ORM in the 'hospital_management' schema. The schema is normalised and uses primary and foreign key constraints to link patients, doctors, departments, appointments, medical records, lab reports, token queues, bills, bill items and pharmacy inventory. Database views (patient_appointments, doctor_schedule_view, medical_record_view, bill_details and inventory_status) simplify common reporting queries, and INSERT → UPDATE → COMMIT transactions keep related records consistent. **JWT authentication**, role-based access control, input validation and error handling protect sensitive hospital data.

Overall, MediDesk reduces paperwork, minimises appointment conflicts, shortens waiting time through token and queue management, improves data accessibility, and improves coordination between patients, doctors and hospital staff, providing a secure, scalable and efficient solution for modern hospital management.

Keywords: Hospital Management, Appointment Booking, OPD Queue, Electronic Medical Records, Node.js, Express.js, React.js, MySQL, Sequelize, JWT, Role-Based Access Control.

INDEX

Chapter 1: Introduction	8
1.1 Background of the Project	8
1.2 Problem Statement	8
1.3 Scope of the Project	8
1.4 Objectives	9
1.5 Importance of the Application	9
1.6 Target Users / Audience.....	9
Chapter 2: System Requirements.....	10
2.1 Hardware Requirements.....	10
2.2 Software Requirements.....	10
2.3 Development Tools and Frameworks	10
Chapter 3: Technology Stack.....	11
3.1 Frontend.....	11
3.2 Backend.....	11
3.3 Database.....	11
3.4 Version Control and Tools.....	11
3.5 Authentication and Security Libraries	11
Chapter 4: System Architecture	13
4.1 High-Level Architecture Diagram	13
4.2 Description of Each Layer	13
4.3 Deployment Architecture.....	14
Chapter 5: Design	15
5.1 Database Design Principles.....	15
5.2 ER Diagram / Database Schema	15
5.3 Views and Transactions	16
5.4 Design Decisions and Assumptions.....	16
Chapter 6: Implementation	17
6.1 Module-wise Implementation	17
6.2 Front-end Logic	17
6.3 API Integration.....	18
6.4 Authentication and Authorization.....	18
Chapter 7: Features	20
7.1 List of Main Features	20
7.2 How Each Feature Works	21
7.3 User Interface Screenshots.....	22
Chapter 8: Testing.....	24

8.1 Postman API Testing	24
8.2 Integration Testing	24
8.3 Tools Used for Testing.....	24
8.4 Test Cases and Results.....	24
Chapter 9: Deployment	26
9.1 Steps to Deploy	26
9.2 Environment Configuration	26
9.3 Hosting of Frontend and Backend	26
Chapter 10: Challenges & Limitations	27
10.1 Issues Faced During Development	27
10.2 Solutions Applied.....	27
10.3 Current Limitations or Known Bugs.....	27
Chapter 11: Future Enhancements	28
11.1 Planned Features	28
11.2 Possible Integrations or Optimizations	28
Chapter 12: Conclusion.....	29
12.1 Summary of the Project	29
12.2 What Was Achieved	29
12.3 Skills Learned	29
12.4 Course Outcome Mapping	29
12.5 Team Contribution	29
Chapter 13: References	30
Chapter 14: Appendices.....	31
Appendix A: Sample Code Snippet	31
Appendix B: Installation and Setup Instructions	31
Appendix C: Default Login Credentials	31
Appendix D: API Reference Summary.....	31
Appendix E: Screenshots	32

Chapter 1: Introduction

1.1 Background of the Project

Hospitals and clinics handle a constant flow of patients, doctors, appointments, prescriptions, laboratory tests, medicines and payments. In many small and mid-sized hospitals this information still lives in paper registers, spreadsheets or separate software for each department. A patient may queue at a registration counter, again at the doctor's room, again at the billing desk and again at the pharmacy, and each desk keeps its own copy of the same details.

MediDesk was built to replace this fragmented way of working with a single, database-backed platform. Every patient, appointment, token, medical record, bill, medicine and lab report is stored in one relational database, and each of the three user roles sees a portal designed for its own tasks.

1.2 Problem Statement

Manual hospital administration leads to long waiting times, inconsistent appointment booking and tracking, data-entry errors and patient records that are difficult to retrieve. Billing, pharmacy and laboratory operations run separately from clinical records, so doctors, patients and administrators cannot see a complete and current picture. The problem is to design and build a secure, centralised web system that lets a hospital register patients, schedule appointments without conflicts, manage the OPD queue, maintain electronic medical records, and connect billing, pharmacy and laboratory data to each patient.

1.3 Scope of the Project

The scope of MediDesk is a **single hospital or clinic** with three roles: Patient, Doctor and Admin. Within this scope the system covers:

- Patient registration and profile management, including patient ID, blood group and insurance details
- Appointment booking based on doctor availability and doctor schedules
- Token and live OPD queue management
- Electronic medical records with diagnosis, vitals and prescriptions
- Billing and insurance, pharmacy stock and expiry tracking, and laboratory reports
- An admin dashboard with hospital-wide KPIs, inventory alerts and audit logs
- Role-based access so that each role sees only its own modules

Multi-hospital chains, telemedicine, mobile applications and online payment gateways are outside the scope of this submission and are listed under future work.

1.4 Objectives

- Centralise hospital operations on one web platform.
- Simplify patient registration and appointment booking.
- Give doctors fast access to electronic medical records.
- Reduce waiting time using token and queue management.
- Link billing, pharmacy and laboratory data to patient records.
- Secure the system with JWT authentication and role-based access control.
- Build a modular, maintainable backend that is easy to extend.

1.5 Importance of the Application

A connected hospital system saves time for everyone involved. Patients spend less time waiting and can see their own records, bills and reports without asking at a counter. Doctors spend less time searching for history and more time treating patients. Administrators get live figures on patients, doctors, revenue, queues and stock, and are warned when essential medicines fall below their reorder threshold, so they can act before a shortage affects care.

1.6 Target Users / Audience

Patients	Doctors	Administrators
Register, book appointments, follow the live OPD token, and view records, bills and diagnostic reports.	Manage today's appointments and the OPD queue, run consultations, and review patient history.	Manage doctors, patients, appointments, pharmacy, lab and billing, and monitor hospital-wide KPIs.

Chapter 2: System Requirements

2.1 Hardware Requirements

Both the Node.js backend and the MySQL database are lightweight for the data volume of a single hospital. A development or demonstration machine with a dual-core processor, 4 GB of RAM (8 GB recommended) and about 1 GB of free disk space is enough to run the frontend, backend and database together on one computer.

2.2 Software Requirements

- Node.js (LTS version) with npm
- MySQL 8.0 or later
- A modern web browser (Chrome, Edge or Firefox)
- Git and a GitHub account for version control
- Visual Studio Code for development
- Postman for API testing; MySQL Workbench for inspecting the schema and data (recommended)

2.3 Development Tools and Frameworks

React.js, bootstrapped with Vite, was used for the frontend. Node.js with Express.js provides the REST API. MySQL is the relational database and Sequelize is the ORM that maps JavaScript models to tables. JSON Web Tokens handle authentication. Postman was used to design and test the APIs, Git and GitHub for source control, and VS Code as the main editor. Twilio is the external SMS service used for notifications.

Chapter 3: Technology Stack

React.js (Vite) Component-based single-page application with three role-based portals and fast development builds.	Node.js + Express.js Non-blocking runtime and a minimal framework for building REST APIs.	MySQL + Sequelize Relational storage with constraints, views and transactions, accessed through an ORM.
--	---	---

3.1 Frontend

The frontend is a single-page application built with React.js and bundled with Vite. It provides three portals (Patient, Doctor and Admin) that share one login page and a common visual design based on cards, a left navigation menu and a top bar showing the signed-in user and role. The login page includes a one-click academic demo sign-in for each role, which makes the system easy to demonstrate. Screens use HTML, CSS and JavaScript at their core, with React managing the component structure.

3.2 Backend

The backend is a REST API built with Node.js and Express.js. It follows a modular architecture: **routes** declare the URLs and the roles allowed to call them, **controllers** hold request handling, validation and business rules, and **models** define the data structures and database access. Authentication middleware, input validation and centralised error handling are applied across the modules.

3.3 Database

MySQL stores all persistent data in the `hospital\_management` schema. Sequelize maps each entity to a model and manages relationships. Referential integrity is enforced with foreign keys and uniqueness and validity rules are enforced with constraints. Views and transactions (Chapter 5) are used for reporting and consistent multi-table updates.

3.4 Version Control and Tools

Git and GitHub were used for version control and collaboration between the two team members. Postman was used to build and run API requests, and npm to manage packages for both the client and server.

3.5 Authentication and Security Libraries

Authentication uses **JSON Web Tokens (JWT)**. After a successful login the server issues a signed token that carries the user's identity and role. Middleware verifies the token on every protected request and checks the role before allowing access to a route. Passwords must be

stored only as salted hashes, never as plain text. Input validation runs before data reaches the database.

Chapter 4: System Architecture

4.1 High-Level Architecture Diagram

MediDesk follows a layered architecture that separates the user interface, the business logic and the stored data. Figure 4.1 shows the layers and how a request moves between them.

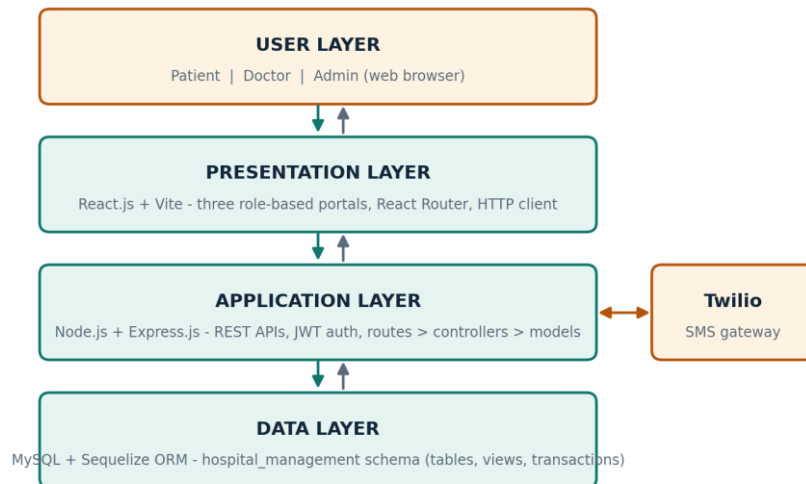


Figure 4.1: Layered architecture of MediDesk

4.2 Description of Each Layer

User layer

Patients, doctors and administrators use the system through a web browser. Each role signs in through the same login page and is taken to its own portal.

Presentation layer (React.js + Vite)

The React single-page application renders the three portals. It never talks to MySQL directly; all data comes from REST API calls to the backend.

Application layer (Node.js + Express.js)

Requests pass through JWT verification and role checks, then into controllers that validate input and apply business rules before reading or writing data through the models.

Data layer (MySQL + Sequelize)

The 'hospital_management' schema stores all entities. Sequelize models, foreign keys, views and transactions keep the data consistent.

External service (Twilio)

Twilio provides SMS notifications. Its configuration is managed from the “Twilio & Settings” page of the admin portal.

4.3 Deployment Architecture

For development and demonstration, the frontend dev server, the Express backend and the MySQL database run as separate local processes on one machine. No container or CI/CD pipeline was configured for this submission. Docker-based deployment is listed under future work (Chapter 11).

Chapter 5: Design

5.1 Database Design Principles

The schema is normalised so that each fact is stored in exactly one place and relationships are expressed through foreign keys instead of duplicated data. Primary and foreign key constraints link every entity, so invalid references (for example, an appointment for a doctor who does not exist) are rejected by the database itself and not only by application code.

5.2 ER Diagram / Database Schema

The core entities are Patient, Doctor, Department, Appointment, Token_Queue, Medical_Record, Lab_Report, Bill, Bill_Item and Pharmacy_Inventory. Figure 5.1 shows how they are related.

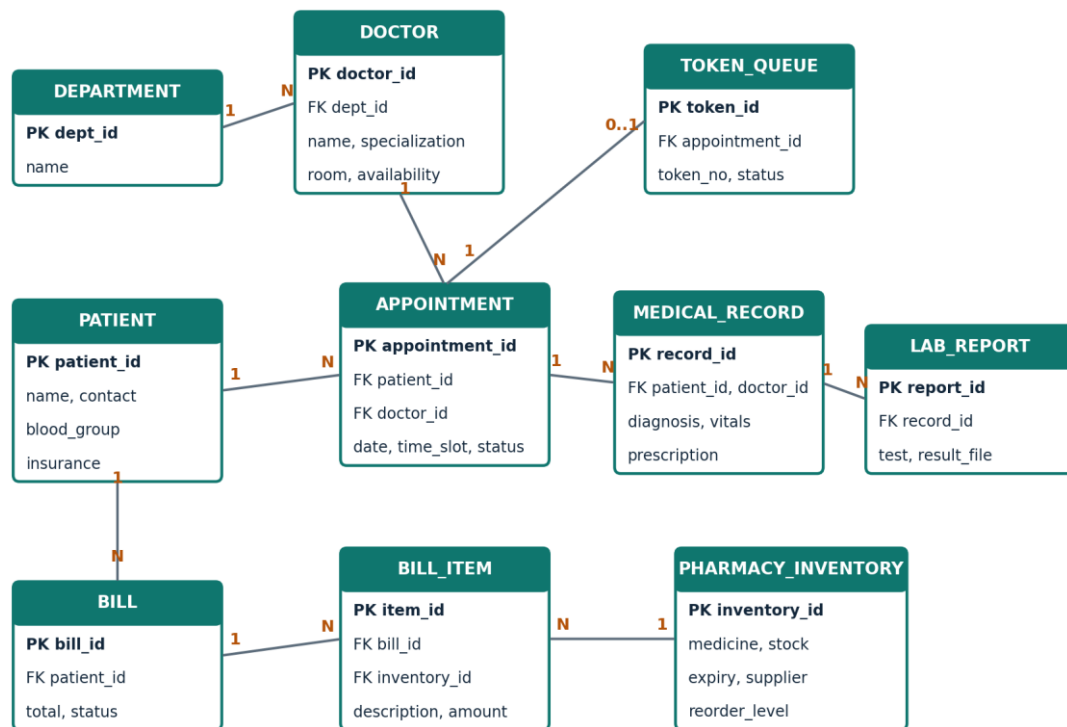


Figure 5.1: Entity-Relationship diagram of the hospital_management schema

Entity	Purpose
Patient	Registered patient profile: patient ID, contact details, blood group and insurance information.
Department / Doctor	Hospital departments and the doctors who belong to them, with specialisation, consultation room and availability.
Appointment	A booking between a patient and a doctor for a date and time slot, with a status such as Confirmed.
Token_Queue	The OPD token issued for an appointment and its progress through the live queue.

Entity	Purpose
Medical_Record	Electronic medical record of a visit: diagnosis, vitals (for example BP and pulse) and prescription.
Lab_Report	Diagnostic tests ordered for a patient and the resulting report files.
Bill / Bill_Item	A patient's invoice and its line items for consultation, medicines and lab tests.
Pharmacy_Inventory	Medicine stock, expiry, supplier and reorder level, used for low-stock alerts.

5.3 Views and Transactions

Five database views were created to simplify reporting queries and to give each portal ready-to-use data:

View	Used for
patient_appointments	Appointments of a patient with doctor and slot details.
doctor_schedule_view	A doctor's schedule and booked slots.
medical_record_view	Visit records joined with patient and doctor information.
bill_details	Bills together with their line items and totals.
inventory_status	Pharmacy stock levels, expiry and reorder status.

Operations that change several tables at once, such as recording a visit and updating the related bill, are run as transactions (INSERT → UPDATE → COMMIT) so that a failure part-way through leaves no half-written data. These transactions were tested and verified.

5.4 Design Decisions and Assumptions

- **One hospital, three roles.** Patient, Doctor and Admin are the only roles. Admin accounts are created by the system owner and not through public registration.
- **Sequelize as the ORM.** Models define the tables and their relationships, which keeps SQL out of the controllers and makes the schema easier to change.
- **Views for reporting.** Frequently needed joins are stored as views so that the API and the reports read from one consistent definition.
- **Separate Bill and Bill_Item.** A bill can hold several items (consultation, medicines, lab tests), so item-level detail is kept in its own table.

Chapter 6: Implementation

6.1 Module-wise Implementation

MediDesk is organised into modules, each with its own routes, controller and model. Figure 6.1 shows how the modules support one complete patient journey.

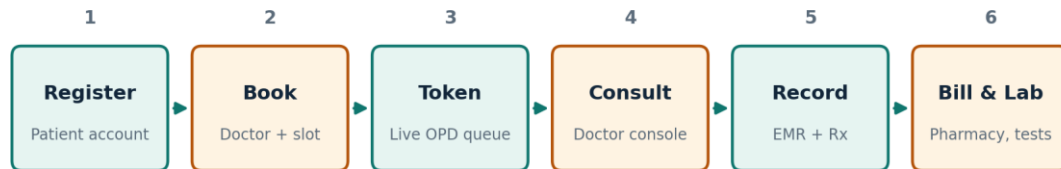


Figure 6.1: End-to-end patient journey handled by MediDesk

Module	Portal	What it does
Authentication	All	Login and registration, JWT issue and verification, role check.
Patient registration	Patient / Admin	Creates patient accounts and profiles with patient ID, blood group and insurance.
Doctors & schedules	Admin / Doctor	Doctor details, departments, rooms, OPD schedule and availability slots.
Appointments	Patient / Doctor / Admin	Book, view and monitor appointments against doctor availability.
Token & OPD queue	All	Issues tokens, shows the live queue, lets the doctor call the next patient.
Electronic records	Doctor / Patient	Diagnosis, vitals and prescriptions per visit.
Billing & insurance	Patient / Admin	Invoices, payments and insurance coverage.
Pharmacy & inventory	Admin	Stock, expiry, suppliers and reorder alerts.
Laboratory	Patient / Admin	Tests and diagnostic report files.
Admin dashboard	Admin	Hospital-wide KPIs, appointment monitoring and audit logs.

6.2 Front-end Logic

The frontend is organised by portal. Each portal has a left navigation menu, a top bar with the notification bell, the signed-in user and a logout button, and a dashboard built from summary cards. The Patient portal has Dashboard, Book Appointment, My Appointments, Live OPD Queue, Medical Records, Bills & Invoices and Diagnostic Reports. The Doctor portal has Doctor Dashboard, Today's Appointments, OPD Queue Manager, Consultation Room, Patient Directory and OPD Schedule. The Admin portal has Overview Dashboard, Doctors & Staff,

Patients Registry, Appointments Monitor, Central OPD Queue, Pharmacy Inventory, Diagnostic Lab, Billing & Insurance, Security Audit Logs and Twilio & Settings.

Because the whole portal is a single-page application, moving between menu items does not reload the page, and the interface stays responsive during a busy OPD session.

6.3 API Integration

The frontend talks to the backend only through REST APIs that exchange JSON. Each request carries the user's JWT, and the backend answers with the requested data or a meaningful HTTP status code (for example 401 for a missing or invalid token and 403 for a role that is not allowed). Each backend module exposes a group of endpoints for the operations of that module.

6.4 Authentication and Authorization

When a user signs in, the server checks the credentials and issues a JWT containing the user's identity and role. Every protected route runs an authentication middleware that verifies the token, and a role check that allows only the intended role. This is why a patient can never open the doctor or admin portal, even by typing a URL. Figure 6.2 shows the login page, which also offers a one-click demo sign-in for each role.

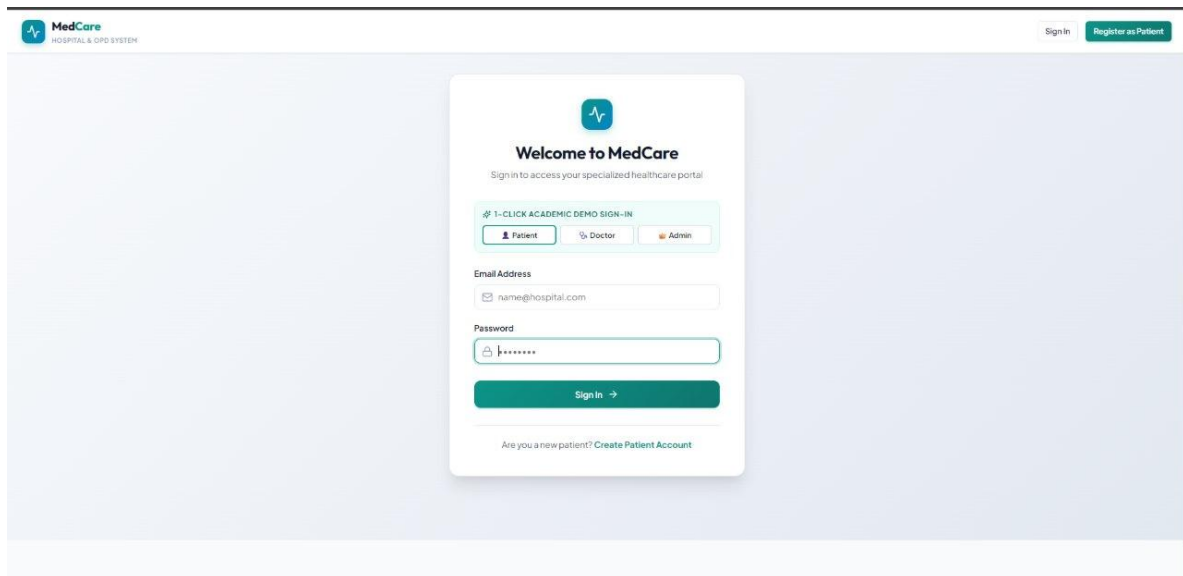


Figure 6.2: Login page with one-click demo sign-in for Patient, Doctor and Admin

```
// Representative example of the JWT + role check used to protect routes
const jwt = require("jsonwebtoken");

exports.protect = (req, res, next) => {
  const token = (req.headers.authorization || "").split(" ")[1];
  if (!token) return res.status(401).json({ message: "Not authenticated" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET); // { id, role }
    next();
  } catch (e) {
    res.status(401).json({ message: "Invalid or expired token" });
  }
}
```

```
    }  
  };  
  
  exports.allowRoles = (...roles) => (req, res, next) =>  
    roles.includes(req.user.role)  
      ? next()  
      : res.status(403).json({ message: "Access denied" });
```

The snippet above illustrates the approach used by the authentication middleware; see the project repository for the exact implementation.

Chapter 7: Features

7.1 List of Main Features

Patient features

- Register and sign in; personal dashboard showing patient ID, blood group and insurance provider
- Book a new appointment with a doctor and see the next scheduled visit with date, time slot and status
- My Appointments: history and upcoming visits
- Live OPD Queue: see the current token
- Medical Records: view diagnosis, vitals (BP, pulse) and prescription status
- Bills & Invoices, including pending invoices that need settlement
- Diagnostic Reports from the laboratory
- Notification bell with unread count

Doctor features

- Clinical console showing today's appointments, the patient currently being served, patients waiting in the queue and consultations completed today
- Manage OPD Queue and call the next patient with one click
- Consultation Room for recording the diagnosis and prescription
- Patient Directory and OPD Schedule

Administrator features

- Executive dashboard with total registered patients, specialist doctors, today's appointments, total hospital revenue, active OPD queue and pending lab reports
- Add doctors and manage staff; view the patients registry
- Appointments monitor listing recent outpatient consultations with appointment number, patient, doctor, specialty, date/slot and status
- Central OPD queue, diagnostic lab and billing & insurance management
- Pharmacy inventory with an alert when essential medicines fall below their safety reorder threshold
- Security audit logs and Twilio SMS settings

7.2 How Each Feature Works

Appointment booking

The patient chooses a doctor and an available time slot. The appointment is saved with a unique appointment number (for example APT-20261001-271) and appears in the patient's dashboard, the doctor's schedule and the admin's appointment monitor.

Token and live OPD queue

Once a patient is in the queue, the doctor's console shows how many are waiting and who is currently being served. Selecting "Call Next Patient" moves the queue forward, and the patient's Live OPD Queue page shows the current token.

Electronic medical record

During a consultation the doctor records the diagnosis, vitals and prescription. The patient can then see the record in their portal, for example a diagnosis with BP and pulse values and a "Rx Prescribed" tag.

Billing and revenue

Bills combine consultation charges, medicines and laboratory tests. Patients see pending invoices that require settlement, and the admin dashboard adds these up as total hospital revenue.

Inventory alerts

The pharmacy inventory stores stock, expiry and reorder levels. When essential medicines fall below their threshold, the admin dashboard shows an inventory alert with a "Restock Inventory" shortcut.

7.3 User Interface Screenshots

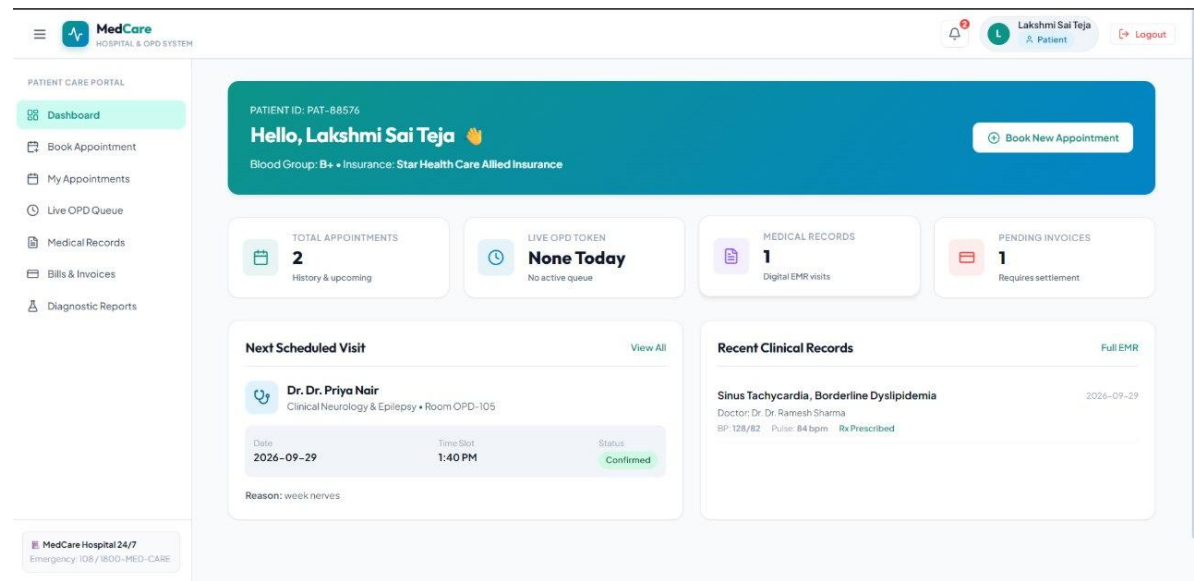


Figure 7.1: Patient portal dashboard

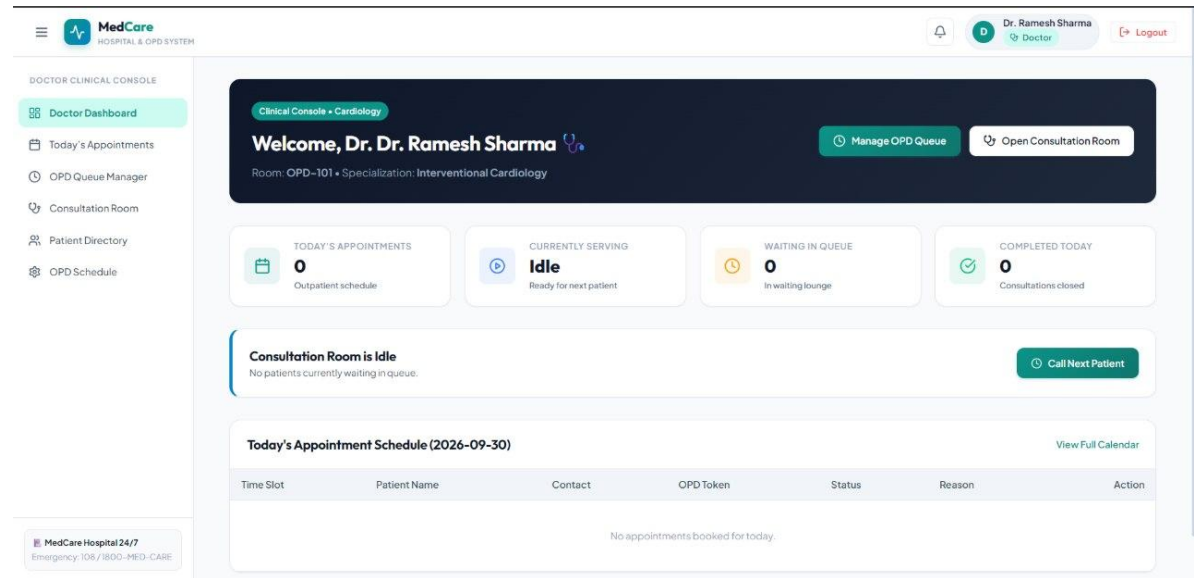


Figure 7.2: Doctor clinical console

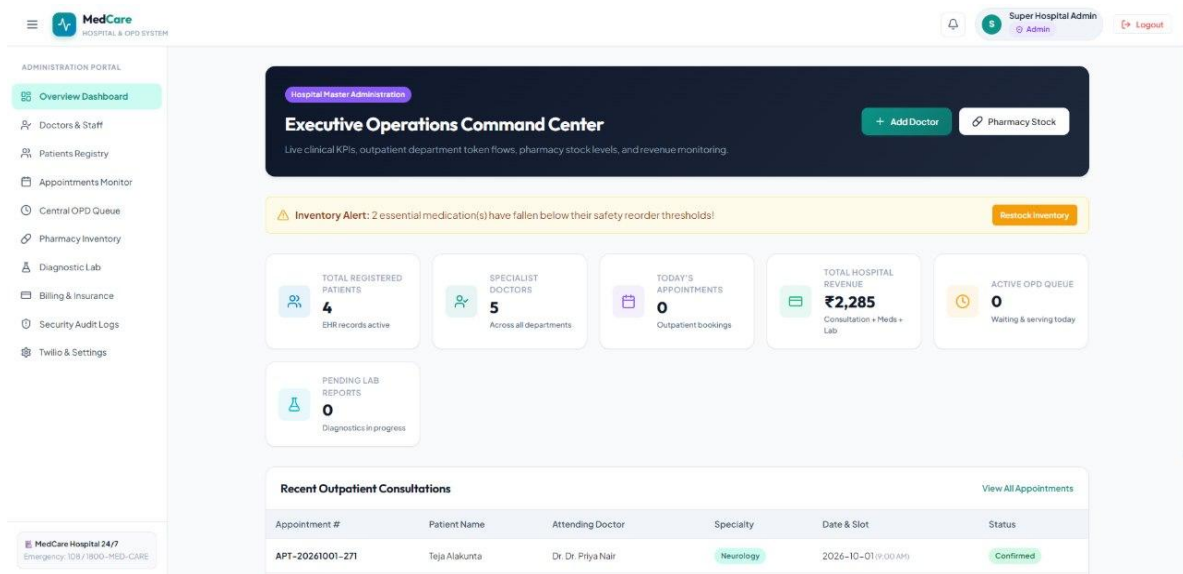


Figure 7.3: Admin executive operations command center

Chapter 8: Testing

8.1 Postman API Testing

Postman was used to design and exercise the REST APIs. Requests are grouped by module and use an environment variable to hold the JWT returned by the login request, so that protected endpoints can be tested in sequence without copying tokens by hand. Both successful requests and deliberate failures (missing token, wrong role, invalid input) were tried.

8.2 Integration Testing

The modules were tested against a running MySQL database and not with mocked data. The three portals were exercised end to end: logging in as each role, opening the dashboard, and checking that the data shown on one portal matches the other (for example, an appointment created for a patient appears in the doctor's schedule and the admin's appointment monitor). The database views and the INSERT → UPDATE → COMMIT transactions were also tested.

8.3 Tools Used for Testing

- Postman: manual API testing
- MySQL Workbench / MySQL client: direct verification of tables, views and constraints
- Web browser: functional testing of the React portals
- Node.js testing tools for backend checks

8.4 Test Cases and Results

The table lists the main test cases. Replace the bracketed results with the outcomes from your own test runs before submission.

#	Test Case	Expected Result	Result
1	One-click demo sign-in as Patient, Doctor and Admin	Each role reaches its own portal dashboard	Pass
2	Login with wrong password	Request rejected with 401	[Pass/Fail]
3	Call a protected API without a token	401 Unauthorized	[Pass/Fail]
4	Patient token opens an admin-only API	403 Forbidden	[Pass/Fail]
5	Register a new patient with invalid or missing fields	Rejected by input validation, with a clear message	[Pass/Fail]
6	Book an appointment for an available slot	Appointment saved and shown in patient, doctor and admin views	[Pass/Fail]
7	Book an already-taken slot	Booking refused	[Pass/Fail]
8	Doctor selects "Call Next Patient"	Queue advances and "Currently Serving" updates	[Pass/Fail]

#	Test Case	Expected Result	Result
9	Doctor saves a diagnosis and prescription	Record appears in the patient's Medical Records	[Pass/Fail]
10	Generate a bill with consultation, medicine and lab items	Bill total equals the sum of its items	[Pass/Fail]
11	Medicine stock falls below reorder level	Admin dashboard shows an inventory alert	[Pass/Fail]
12	Query each database view	Views return correctly joined data	Pass
13	Run INSERT → UPDATE → COMMIT transaction	All changes saved together; nothing partial on failure	Pass
14	Patient opens the doctor or admin URL directly	Access denied / redirected	[Pass/Fail]

Chapter 9: Deployment

9.1 Steps to Deploy

- Install Node.js, npm and MySQL, then create the `hospital\_management` database.
- Load the schema and seed data (roles, departments, sample doctors, medicines).
- Configure the server environment variables (Section 9.2).
- Start the backend: open the server folder, run `npm install`, then `npm run dev`.
- Start the frontend: open the client folder, run `npm install`, then `npm run dev`.
- Open the printed local address in a browser and sign in using a demo account.

9.2 Environment Configuration

Configuration is kept in a `.env` file that is never committed to Git. It holds the database connection details (host, port, user, password and database name), the JWT secret and expiry, the server port, and the Twilio credentials. A `.env.example` file with placeholder values documents the required variables without exposing real secrets.

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=hospital_management
DB_USER=<your-mysql-user>
DB_PASSWORD=<your-mysql-password>
JWT_SECRET=<long-random-string>
JWT_EXPIRES_IN=1d
PORT=<server-port>
TWILIO_ACCOUNT_SID=<optional>
TWILIO_AUTH_TOKEN=<optional>
```

9.3 Hosting of Frontend and Backend

For this submission the frontend, backend and database run locally, which is suitable for development and a classroom demonstration. A hosted deployment (for example, containers with Docker, a cloud VM or a managed MySQL service) is part of future work.

Chapter 10: Challenges & Limitations

10.1 Issues Faced During Development

- **Keeping many modules consistent.** Appointments, queue, records, billing, pharmacy and lab all refer to the same patient, so relationships had to be designed carefully to avoid orphaned or contradictory data.
- **Role-based navigation and access.** Three portals with different menus and permissions had to share one login flow and one backend.
- **Multi-table updates.** Actions such as completing a consultation affect several tables, and a failure part-way could leave inconsistent data.
- **Team coordination.** Two developers worked on the same codebase and needed clear branches and merges on GitHub.

10.2 Solutions Applied

- A normalised schema with primary and foreign keys, plus views for common joins.
- JWT authentication with a role check in middleware, and role-specific menus in the frontend.
- Transactions (INSERT → UPDATE → COMMIT) so related changes succeed or fail together.
- A modular routes/controllers/models structure and Git-based workflow so both members could work in parallel.

10.3 Current Limitations or Known Bugs

- The doctor name appears with a duplicated title (“Dr. Dr. Ramesh Sharma”) in several screens, because the title is stored in the name and added again by the interface. This is a display issue only and can be fixed by removing the extra prefix.
- SMS notifications through Twilio depend on valid credentials and are not part of the local demo.
- The system is designed for a single hospital and has been run only in a local environment.
- There is no telemedicine, analytics dashboard or online payment gateway yet.

Chapter 11: Future Enhancements

11.1 Planned Features

SMS notifications Full appointment and queue notifications through Twilio.	Analytics dashboard Charts for patient load, revenue, department utilisation and waiting time.	Telemedicine Video consultation for follow-up visits.
--	--	---

11.2 Possible Integrations or Optimizations

- Docker deployment so the frontend, backend and database can be started with one command.
- Online payment gateway for bills and insurance claims.
- Automated tests (unit and API tests) run through a CI pipeline.
- Caching of frequently requested data such as doctor availability.
- Mobile application for patients.

Chapter 12: Conclusion

12.1 Summary of the Project

MediDesk replaces paper-based and disconnected hospital processes with one platform that serves patients, doctors and administrators. Using React.js, Node.js, Express.js, MySQL and Sequelize, the team designed a normalised database, a modular REST backend and three role-based portals with dashboards.

12.2 What Was Achieved

- One connected platform for patients, doctors and admins
- Role-based portals with dashboards
- A normalised schema with primary and foreign keys, five views and verified transactions
- A secure, modular REST backend with JWT authentication and role-based access control

12.3 Skills Learned

- Relational database design, normalisation, views and transactions
- Building REST APIs with Node.js, Express.js and Sequelize
- JWT authentication and role-based authorisation
- Building a multi-portal React frontend with Vite
- API testing with Postman and collaboration with Git and GitHub

12.4 Course Outcome Mapping

CO	Course Outcome	Where it appears in this project
CO1	[Course outcome text]	Database design: ER model, normalisation, constraints (Chapter 5)
CO2	[Course outcome text]	Views and transactions on the hospital_management schema (Section 5.3)
CO3	[Course outcome text]	REST backend with Node.js, Express.js and Sequelize (Chapters 3 and 6)
CO4	[Course outcome text]	JWT authentication, role-based access, testing and deployment (Chapters 6, 8 and 9)

12.5 Team Contribution

Student	Roll Number	Contribution
P. Rahul	2520090016	[Modules and tasks handled]
A. Lakshmi Sai Teja	2520080005	[Modules and tasks handled]

Chapter 13: References

- React: Official Documentation, <https://react.dev>
- Vite: Official Documentation, <https://vitejs.dev>
- Node.js: Official Documentation, <https://nodejs.org/en/docs>
- Express.js: Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- Sequelize: Official Documentation, <https://sequelize.org>
- jsonwebtoken (npm), <https://www.npmjs.com/package/jsonwebtoken>
- Twilio SMS Documentation, <https://www.twilio.com/docs/sms>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs: JavaScript and Web APIs, <https://developer.mozilla.org>

Chapter 14: Appendices

Appendix A: Sample Code Snippet

A view definition of the kind used to simplify reporting queries (columns shown are illustrative):

```
CREATE VIEW patient_appointments AS
SELECT a.appointment_id, p.name AS patient_name,
       d.name AS doctor_name, a.appointment_date, a.time_slot, a.status
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id;
```

A transaction that keeps related changes together:

```
START TRANSACTION;
  INSERT INTO medical_records (patient_id, doctor_id, diagnosis) VALUES (?, ?, ?);
  UPDATE appointments SET status = "Completed" WHERE appointment_id = ?;
COMMIT;
```

Appendix B: Installation and Setup Instructions

- Create the MySQL database `hospital\_management` and load the schema and seed data.
- Copy `.env.example` to `.env` in the server folder and fill in the values.
- In the server folder run `npm install` and `npm run dev`.
- In the client folder run `npm install` and `npm run dev`.
- Open the local address shown in the terminal and sign in.

Appendix C: Default Login Credentials

The login page provides one-click demo sign-in for each role. Seeded demo accounts:

Role	Email	Password
Patient	[patient email]	[password]
Doctor	[doctor email]	[password]
Admin	[admin email]	[password]

Appendix D: API Reference Summary

Resource	Purpose	Allowed roles
Auth	Register, login, current user	Public / All

Resource	Purpose	Allowed roles
Patients	Profiles and patient registry	Patient (own), Admin, Doctor (read)
Doctors & departments	Doctor details, specialisation, schedule	Admin (manage), all (read)
Appointments	Book, list, update status	Patient, Doctor, Admin
Token / OPD queue	Issue tokens, call next, queue status	Doctor, Admin, Patient (read)
Medical records	Diagnosis, vitals, prescriptions	Doctor (write), Patient (read own)
Billing & insurance	Bills, items, payments	Admin, Patient (read own)
Pharmacy inventory	Stock, expiry, reorder alerts	Admin
Lab reports	Tests and report files	Admin, Doctor, Patient (read own)

Full request and response examples are in the accompanying Postman collection.

Appendix E: Screenshots

Figures 6.2, 7.1, 7.2 and 7.3 show the login page and the Patient, Doctor and Admin dashboards. Additional screenshots (booking form, OPD queue manager, pharmacy inventory, MySQL Workbench schema) can be added below before submission.